



UNIVERSITI
TEKNOLOGI
PETRONAS

RBB4633 Next Generation Computing Security (May 2026)

Lab 1 – Adversarial Attack Using FGSM in PyTorch

Implementing Adversarial Example Attack Using Fast Gradient Sign Method (FGSM)

Name	Muhammad Eusoff Amir bin Hazri
Student ID	22010491
Course	Computer Engineering
Date	4th June 2026

INTRODUCTION

This lab is about implementing an adversarial attack using the Fast Gradient Sign Method (FGSM) in PyTorch. Adversarial attacks basically is just adding noise to the original output image into a blurrier image. FGSM is one of the easiest ways to do this by using the gradient of the loss function to modify the input image in a direction that maximizes the model's error.

$$x_{adv} = x + \epsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

Where:

- x = original input
- ϵ = perturbation magnitude
- J = loss function
- θ = model parameters

Where x is our original clean input image, ϵ is the magnitude parameter controlling the attack or noise strength, and $J(\theta, x, y)$ represents the loss function of the model.

METHODS AND MODELS USED

Dataset Setup

We used the MNIST dataset, which contains 28 x 28 grayscale images of handwritten digits from 0 to 9. We loaded it using `torchvision.datasets.MNIST` and turned the images into tensors using `transforms.ToTensor()`. The DataLoader was set with a batch size of 1 so we could generate and analyze adversarial samples one by one.

```
20 # =====
21 # Task 2 - Load MNIST Dataset
22 # =====
23 transform = transforms.ToTensor()
24
25 test_dataset = torchvision.datasets.MNIST(
26     root='./data',
27     train=False,
28     transform=transform,
29     download=True
30 )
31
32 test_loader = torch.utils.data.DataLoader(
33     dataset=test_dataset,
34     batch_size=1,
35     shuffle=True
36 )
37
```

Model Architecture

The script uses a basic feed-forward fully connected neural network class named SimpleNN. Its architecture goes as follows:

- **Flattening:** Inside forward(), the 28x28 image matrix is reshaped into a flat 784-dimensional feature vector using `x.view(-1, 28 * 28)`.
- **Hidden Layer:** A fully connected linear mapping (`nn.Linear(784, 128)`) that handles the primary input, followed by a non-linear `torch.relu` activation.
- **Output Layer:** A final linear output layer (`nn.Linear(128, 10)`) that maps the hidden features down to raw confidence scores across the 10 target digit classes.

```
39 # =====
40 # Task 3 - Define SimpleNN
41 # (REQUIRED when loading model)
42 # =====
43 class SimpleNN(nn.Module):
44     def __init__(self):
45         super(SimpleNN, self).__init__()
46
47         self.fc1 = nn.Linear(28 * 28, 128)
48         self.fc2 = nn.Linear(128, 10)
49
50     def forward(self, x):
51         x = x.view(-1, 28 * 28)
52         x = torch.relu(self.fc1(x))
53         x = self.fc2(x)
54         return x
55
```

Running the FGSM Attack

1. We loaded the pre-trained weights file `simple_nn_mnist.pth` and set the network to evaluation mode using `model.eval()`.
2. A single data sample (image, label) was fetched, and we explicitly turned on gradient tracking for the input tensor via `image.requires_grad = True`.
3. We ran a forward pass to compute predictions and used Cross-Entropy Loss (`nn.CrossEntropyLoss()`) to find the error against the true label.
4. Running `loss.backward()` computed the gradients back to the input layers, letting us isolate the input data gradient matrix via `image.grad.data`.

5. The fgsm_attack function extracted the sign of these gradients, multiplied them by a scalar value ($\epsilon = 0.2$), and added this structural noise to the original picture.
6. Finally, torch.clamp(perturbed_image, 0, 1) was used to make sure the modified pixel values stayed inside valid bounds.

```
# =====  
# Task 4 - Load trained model  
# =====  
model = SimpleNN().to(device)  
  
model.load_state_dict(  
    torch.load("simple_nn_mnist.pth", map_location=device)  
)  
  
model.eval()  
  
print("Model loaded successfully!")
```

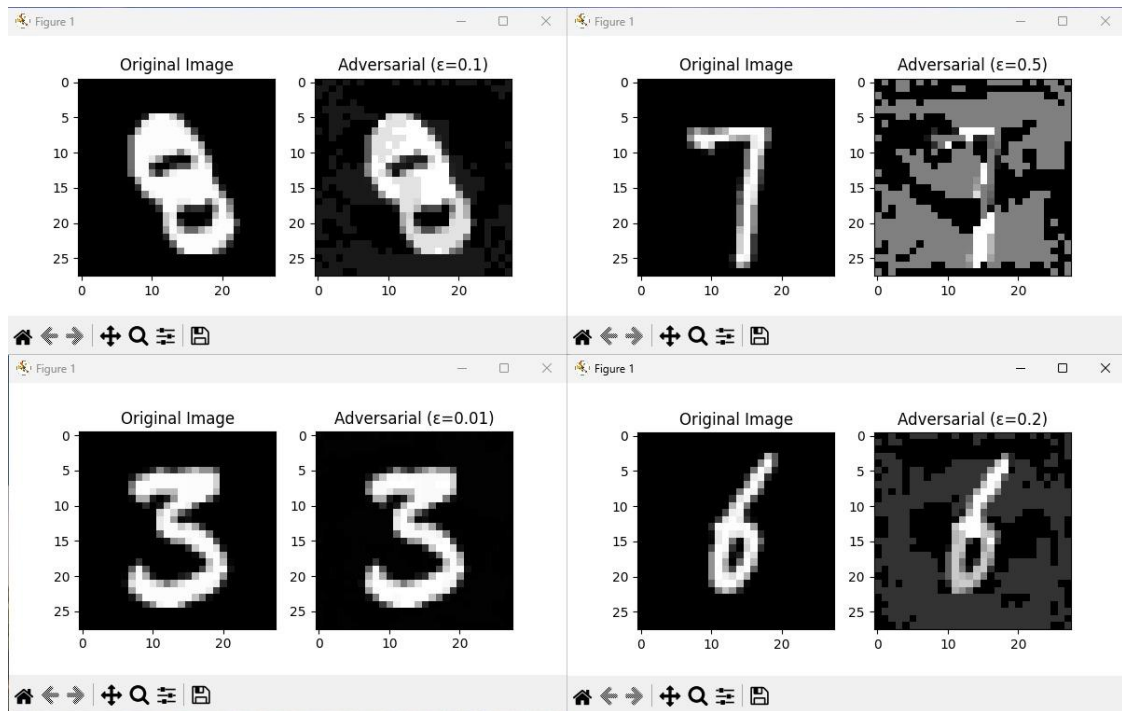
```
# =====  
# Task 5 - FGSM Attack Function  
# =====  
loss_fn = nn.CrossEntropyLoss()  
  
def fgsm_attack(image, epsilon, data_grad):  
    sign_grad = data_grad.sign()  
    perturbed_image = image + epsilon * sign_grad  
    perturbed_image = torch.clamp(perturbed_image, 0, 1)  
    return perturbed_image
```

RESULTS & ANALYSIS

Evaluation Outputs

Running the script produced the following output data in the terminal and visualization plots:

```
PS D:\Users\utp\Desktop\lab1-NGCS> py main.py  
Model loaded successfully!  
True Label: 7  
Original Prediction: 7  
Adversarial Prediction: 5  
█
```



Analysis

The model confidently correctly classified the original image as a "7". However, when we introduced adversarial noise using a step size of $\epsilon = 0.2$, the prediction flipped entirely to a "5". Looking at the generated plot outputs, the texturing changes are visible as a light pixel grain or background gray tint across the image canvas. Even though a human can easily tell that the drawing is still a digit 7, the model gets completely thrown off by this noise pattern because it alters the pixel feature distributions that the neural network layers use to make classifications.

As shown in our visualization check:

Original Image	Epsilon, ϵ	Adversarial Image
3	0.01	Similar
8	0.1	Slightly Blurry
7	0.2	Blurry
6	0.5	Very Blurry

In conclusion, the higher the value of epsilon (ϵ), the blurrier the adversarial Image

Discussion Questions & Answers

1. What is the purpose of FGSM?

FGSM creates small changes (noise) in input data (original image) to trick a model into making wrong predictions. It is primarily used to discover hidden vulnerabilities and assess security risks in deep learning systems.

2. Why are adversarial examples dangerous in autonomous systems?

In systems like self-driving cars or medical AI, even tiny changes can cause serious mistakes. For example, a vehicle's computer vision system could misread a stop sign or completely fail to detect pedestrians crossing the road, which leads to major real-world accidents.

3. How does epsilon affect attack strength?

- Small ϵ : adversarial output stays more similar to the original picture with minimal noise, making it harder for humans to spot but less likely to fool the model.
- Large ϵ : adversarial output that is less similar to the original input picture with a lot of visible noise from the attack. This makes the attack much stronger at breaking the model's accuracy and is visible to the human eye.

4. Suggest one defense mechanism against adversarial attacks.

One common defense is adversarial training. In this approach, the model is trained using a combination of the original dataset and the adversarial data (output data) generated from running the script. Re-training the model using this outputted data helps it learn how to look past the malicious noise and make accurate predictions.

Conclusion

This experiment successfully showed how easily a standard feed-forward network can be tricked using white-box gradient operations like FGSM. By making minor changes in the direction of the loss gradient, the script successfully caused our model to misclassify a clear digit "7" as a "5". These findings emphasize that high performance on a regular test set does not mean an AI model is safe. It shows why implementing security defenses like adversarial training is essential before launching models in real-world engineering or software deployments.

SOURCE CODE

```
# =====
# Adversarial_06_FGSM_Attack.py
# Purpose:
# Use trained model + perform FGSM attack
# =====

import torch
import torch.nn as nn
import torchvision
import torchvision.transforms as transforms
import matplotlib.pyplot as plt

# =====
# Task 1 - Device setup
# =====
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# =====
# Task 2 - Load MNIST Dataset
# =====
transform = transforms.ToTensor()

test_dataset = torchvision.datasets.MNIST(
    root='./data',
    train=False,
    transform=transform,
    download=True
)

test_loader = torch.utils.data.DataLoader(
    dataset=test_dataset,
    batch_size=1,
    shuffle=True
)

# =====
# Task 3 - Define SimpleNN
# (REQUIRED when loading model)
# =====
class SimpleNN(nn.Module):
    def __init__(self):
        super(SimpleNN, self).__init__()
```

```

        self.fc1 = nn.Linear(28 * 28, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = x.view(-1, 28 * 28)
        x = torch.relu(self.fc1(x))
        x = self.fc2(x)
        return x

# =====
# Task 4 - Load trained model
# =====
model = SimpleNN().to(device)

model.load_state_dict(
    torch.load("simple_nn_mnist.pth", map_location=device)
)

model.eval()

print("Model loaded successfully!")

# =====
# Task 5 - FGSM Attack Function
# =====
loss_fn = nn.CrossEntropyLoss()

def fgsm_attack(image, epsilon, data_grad):
    sign_grad = data_grad.sign()
    perturbed_image = image + epsilon * sign_grad
    perturbed_image = torch.clamp(perturbed_image, 0, 1)
    return perturbed_image

# =====
# Task 6 - Generate Adversarial Example
# =====
image, label = next(iter(test_loader))

image, label = image.to(device), label.to(device)

image.requires_grad = True

# Forward pass
output = model(image)
init_pred = output.argmax(dim=1)

```



```

# Loss
loss = loss_fn(output, label)

model.zero_grad()
loss.backward()

# Gradient
data_grad = image.grad.data

# FGSM attack
epsilon = 0.2
adv_image = fgsm_attack(image, epsilon, data_grad)

# Re-check prediction
adv_output = model(adv_image)
adv_pred = adv_output.argmax(dim=1)

# =====
# Task 7 - Results
# =====
print("True Label:", label.item())
print("Original Prediction:", init_pred.item())
print("Adversarial Prediction:", adv_pred.item())

# =====
# Task 8 - Visualization
# =====
plt.figure(figsize=(6,3))

plt.subplot(1,2,1)
plt.imshow(image.squeeze().detach().cpu().numpy(), cmap='gray')
plt.title("Original Image")

plt.subplot(1,2,2)
plt.imshow(adv_image.squeeze().detach().cpu().numpy(), cmap='gray')
plt.title(f"Adversarial ( $\epsilon$ ={{epsilon}})")

plt.show()

```